

Spring Cloud Gateway 中 dependencies 模块与父项目继承的区别

你这个理解非常对！从“版本统一”的效果上看，用「继承父项目」和

「`dependencyManagement` 依赖管理」确实都能做到，但 Spring Cloud Gateway 用这种 `spring-cloud-gateway-dependencies` 模块的方式，是为了解决更复杂的场景，而不是简单的版本统一。

下面我给你把两种方式的区别讲透，你就明白为什么官方这么设计了。

一、先回答你的核心问题

直接让 4 个项目继承父项目，在父项目中定义版本，行不行？

✅ 在你的场景里，完全可以实现“版本一致”这个目标。

比如你可以这么写：

代码块

```
1 <!-- 父 pom.xml -->
2 <groupId>org.springframework.cloud</groupId>
3 <artifactId>spring-cloud-gateway-parent</artifactId>
4 <version>4.1.0</version>
5
6 <properties>
7     <gateway.version>4.1.0</gateway.version>
8 </properties>
```

子模块继承：

代码块

```
1 <!-- 子模块 pom.xml -->
2 <parent>
3     <groupId>org.springframework.cloud</groupId>
4     <artifactId>spring-cloud-gateway-parent</artifactId>
5     <version>4.1.0</version>
6 </parent>
7
8 <dependencies>
```

```
9      <dependency>
10          <groupId>org.springframework.cloud</groupId>
11          <artifactId>spring-cloud-gateway-webflux</artifactId>
12          <version>${gateway.version}</version>
13      </dependency>
14  </dependencies>
```

这样所有子模块的版本也能统一，和 `dependencyManagement` 效果是一样的。

二、那为什么 Spring Cloud 要单独弄一个 `dependencies` 模块？

官方这么做，主要是为了给外部用户提供“BOM（Bill of Materials）依赖管理”，而不仅仅是给内部子模块用。

1. 给外部用户用：让用户不用写版本号

你作为外部开发者，使用 Spring Cloud Gateway 时，通常会这么写：

代码块

```
1  <!-- 你的项目 pom.xml -->
2  <dependencyManagement>
3      <dependencies>
4          <dependency>
5              <groupId>org.springframework.cloud</groupId>
6              <artifactId>spring-cloud-gateway-dependencies</artifactId>
7              <version>4.1.0</version>
8              <type>pom</type>
9              <scope>import</scope>
10         </dependency>
11     </dependencies>
12 </dependencyManagement>
13
14 <dependencies>
15     <!-- 不用写版本号! -->
16     <dependency>
17         <groupId>org.springframework.cloud</groupId>
18         <artifactId>spring-cloud-starter-gateway</artifactId>
19     </dependency>
20 </dependencies>
```

这里的关键是：

- `spring-cloud-gateway-dependencies` 是一个可以被外部项目 `import` 的 POM

- 外部项目不需要继承 Spring Cloud 的父项目，也能统一版本
- 这是 Maven 中 “BOM 依赖管理” 的标准用法

如果用 “继承父项目” 的方式，外部用户必须：

代码块

```
1 <parent>
2   <groupId>org.springframework.cloud</groupId>
3   <artifactId>spring-cloud-gateway-parent</artifactId>
4   <version>4.1.0</version>
5 </parent>
```

但用户自己的项目通常已经有一个父项目了，**无法多重继承**，所以这种方式对外部用户非常不友好。

2. 内部子模块也可以不继承父项目，依然统一版本

在 Gateway 内部，子模块可以不继承父 POM，直接 `import` 这个 dependencies 模块：

代码块

```
1 <dependencyManagement>
2   <dependencies>
3     <dependency>
4       <groupId>org.springframework.cloud</groupId>
5       <artifactId>spring-cloud-gateway-dependencies</artifactId>
6       <version>${project.version}</version>
7       <type>pom</type>
8       <scope>import</scope>
9     </dependency>
10  </dependencies>
11 </dependencyManagement>
```

这样既实现了版本统一，又保留了模块的灵活性。

3. 核心区别： `继承` vs `import`

对比项	继承父项目 (parent)	import dependencies (BOM)
-----	----------------	---------------------------

适用场景	内部项目，只有一个父项目	内部 + 外部，支持 “无侵入” 引入
外部用户	必须继承，无法与自己的父项目共存	只需 <code>import</code> ，不影响自己的项目结构
传递性	继承所有父项目的依赖和配置	仅导入 <code>dependencyManagement</code> 中的版本号
灵活性	差（只能继承一个）	高（可以 <code>import</code> 多个 BOM）
Spring 官方推荐	内部项目使用	对外提供的组件 / 框架，必须用 BOM

三、总结

1. 在你的场景里，用 “继承父项目” 完全可以实现版本统一，效果是一样的。
2. Spring Cloud Gateway 用 `spring-cloud-gateway-dependencies` 模块，是为了给外部用户提供无侵入的 BOM 依赖管理。
3. 这种设计是 Maven 生态的标准做法，Spring Boot、Spring Cloud、Apache Commons 等几乎所有开源框架都这么用。

下面给你一套企业级可直接落地、完全对标 Spring 官方的「自定义 BOM 依赖管理」完整配置，无需继承父工程、无侵入、统一版本，适配多模块项目，可直接复制使用。

四、实战：自定义项目 BOM 依赖管理（完整可运行）

整体工程结构

遵循 Spring 官方架构，拆分父工程、BOM版本管理模块、业务子模块，结构清晰、解耦彻底：

代码块

```
1  your-project-root（根工程，仅打包聚合）
2  |—— your-project-bom（BOM版本统一模块，核心：管理所有依赖版本）
3  |—— project-module-a（业务子模块1）
4  |—— project-module-b（业务子模块2）
5  |—— project-module-c（业务子模块3）
```

1. 根工程 pom.xml（聚合工程，不做版本管控）

根工程只负责聚合所有子模块，不定义依赖版本、不统一配置，保持纯净：

代码块

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4          xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5          http://maven.apache.org/xsd/maven-4.0.0.xsd">
6
7      <groupId>com.xxx</groupId>
8      <artifactId>your-project-root</artifactId>
9      <version>1.0.0</version>
10     <packaging>pom</packaging>
11     <name>项目根聚合工程</name>
12
13     <modules>
14         <module>your-project-bom</module>
15         <module>project-module-a</module>
16         <module>project-module-b</module>
17         <module>project-module-c</module>
18     </modules>
19
20     <!-- 仅统一编译编码、JDK版本，无依赖、无版本号 -->
21     <properties>
22         <maven.compiler.source>17</maven.compiler.source>
23         <maven.compiler.target>17</maven.compiler.target>
24         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
25     </properties>
26
27 </project>
```

2. BOM核心模块 pom.xml（版本统一核心）

单独创建 **your-project-bom** 模块，所有依赖版本全部在这里统一管控，对外提供版本能力，子模块、外部项目均可导入：

代码块

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4          xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5          http://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
5      <parent>
6          <groupId>com.xxx</groupId>
7          <artifactId>your-project-root</artifactId>
8          <version>1.0.0</version>
9      </parent>
10
11      <modelVersion>4.0.0</modelVersion>
12      <artifactId>your-project-bom</artifactId>
13      <packaging>pom</packaging>
14      <name>项目统一版本BOM管理</name>
15
16      <!-- 核心：只管理版本，不实际引入依赖 -->
17      <dependencyManagement>
18          <dependencies>
19              <!-- 统一管控自定义模块版本 -->
20              <dependency>
21                  <groupId>com.xxx</groupId>
22                  <artifactId>project-module-a</artifactId>
23                  <version>${project.version}</version>
24              </dependency>
25              <dependency>
26                  <groupId>com.xxx</groupId>
27                  <artifactId>project-module-b</artifactId>
28                  <version>${project.version}</version>
29              </dependency>
30              <dependency>
31                  <groupId>com.xxx</groupId>
32                  <artifactId>project-module-c</artifactId>
33                  <version>${project.version}</version>
34              </dependency>
35
36              <!-- 统一管控第三方依赖版本（可无限扩展） -->
37              <dependency>
38                  <groupId>org.springframework.cloud</groupId>
39                  <artifactId>spring-cloud-gateway-starter</artifactId>
40                  <version>4.1.0</version>
41              </dependency>
42              <dependency>
43                  <groupId>org.springframework.boot</groupId>
44                  <artifactId>spring-boot-starter</artifactId>
45                  <version>3.2.0</version>
46              </dependency>
47          </dependencies>
48      </dependencyManagement>
49
50  </project>
```

3. 业务子模块引用（无版本、无继承）

所有业务子模块 **无需继承任何自定义父工程**，只需导入BOM，即可免写版本号，彻底解耦：

代码块

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4          xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
5      <parent>
6          <groupId>org.springframework.boot</groupId>
7          <artifactId>spring-boot-starter-parent</artifactId>
8          <version>3.2.0</version>
9          <relativePath/>
10     </parent>
11
12     <modelVersion>4.0.0</modelVersion>
13     <artifactId>project-module-a</artifactId>
14     <name>业务模块A</name>
15
16     <!-- 导入自定义BOM，统一所有版本（核心代码） -->
17     <dependencyManagement>
18         <dependencies>
19             <dependency>
20                 <groupId>com.xxx</groupId>
21                 <artifactId>your-project-bom</artifactId>
22                 <version>1.0.0</version>
23                 <type>pom</type>
24                 <scope>import</scope>
25             </dependency>
26         </dependencies>
27     </dependencyManagement>
28
29     <!-- 引入依赖，无需写版本号 -->
30     <dependencies>
31         <dependency>
32             <groupId>com.xxx</groupId>
33             <artifactId>project-module-b</artifactId>
34         </dependency>
35         <dependency>
36             <groupId>org.springframework.cloud</groupId>
37             <artifactId>spring-cloud-gateway-starter</artifactId>
38         </dependency>
39     </dependencies>
40
```

五、该方案核心优势（对标Spring官方）

1. **无侵入、不冲突**：子模块可自由继承Spring Boot官方父工程，无需绑定自定义父工程，解决Maven单继承限制
2. **版本全局统一**：所有模块、第三方依赖版本集中在BOM模块管理，修改一处、全局生效，杜绝版本混乱
3. **支持外部引用**：其他外部项目可直接import当前BOM，快速适配项目版本，无需同步维护版本
4. **职责单一清晰**：根工程负责聚合、BOM负责版本、子模块负责业务，完全遵循Spring官方设计思想

六、使用规范

- 所有版本号只允许在bom模块中定义，子模块、业务代码禁止写版本号
- 新增依赖、升级版本，仅修改 `your-project-bom` 模块即可
- 多团队协作时，统一依赖基线，避免本地版本差异导致的编译、运行异常

（注：文档部分内容可能由 AI 生成）